基于本体的领域知识工程实践

——以金融测试为例

演讲嘉宾：杨冬瑞

目录

CONTENT

01 金融测试难在哪

02 为什么选择本体

03 测试工艺 × 分层本体

04 从影响分析到案例生成

05 本体是手段不是目的



01 金融测试难在哪

关于我：从技术研发到金融测试 AI 赋能

杨冬瑞

江苏润和软件股份有限公司 · 人工智能中心

东南大学

导航制导与控制专业博士

华为

- hyperViser 开发
- 性能分析平台
- AI 辅助模型驱动开发工具
- 嵌入式 Agent

润和软件

金融测试的 AI 赋能

实践地图

起点：金融测试项目





本体平台

业务知识与变更影响



测试工艺平台

从需求到案例



测试执行平台

APP 操作与执行

先各自解决具体问题，再从应用里归纳本体

一条金融测试案例的“一生”

交易一致性 | 超时专项　　真实案例（脱敏）

测试内容：本机构处理超时，钱包 APP 断开连接

① 前置：本机构或对手机构处理时间较长

② APP 发起，本机构向对手机构发一笔报文

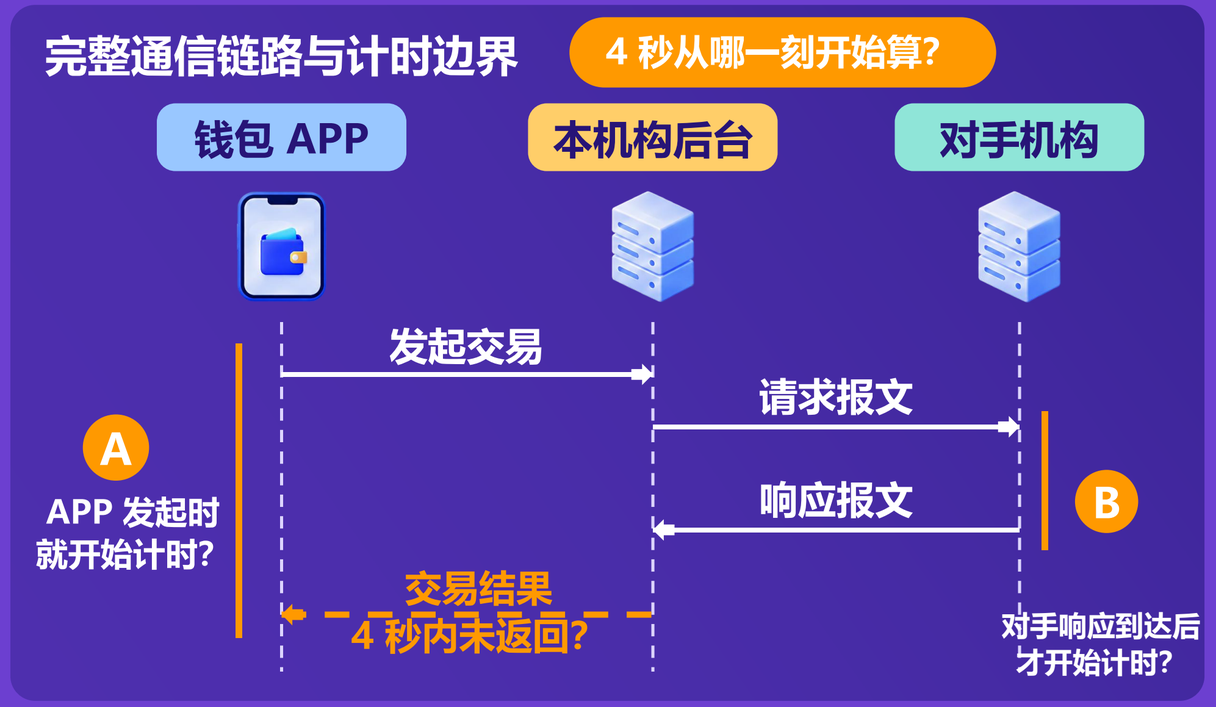
③ 本机构 4 秒内未把结果返回 APP

④ 钱包 APP 显示该笔交易超时

⑤ 用户进入交易记录查看最终结果

“较长”——多长才算长?

4 秒内不返回——这个异常怎么造?



案例理解 → 数据准备 → 跨系统操作 → 日志查询 → 报文提取 → 挡板制作 → 结果比对 → 证据回填

- 每个部分都可能引入经验或关联知识

复杂度 = 业务规则 × 系统关系 × 专家经验 × 历史资产

测试的难点在知识：缺、贵、杂、隐

缺 业务知识难共享

测试和开发往往由不同厂商参与，需求背后的规则和系统关系难以共享



贵 获取成本高

文档散、老员工忙，联调记录翻不到



杂 关联系统复杂

APP、本机构、对手机构、挡板、日志，一个案例牵动多方





隐 经验信息多

“较长”多长、异常怎么造、结果怎么判，都没写下来



引入本体：把判断依据变成知识

1 可关联

对象—关系—规则，把需求连到系统和案例



2 可追溯

每条知识带来源和版本，结论讲得清依据



3 可复用

一次澄清，下次直接用；人走知识在



运维、客服、合规等知识密集领域同理

让跨系统的知识能够沉淀、关联和复用，支撑具体测试任务



02 为什么选择本体

为什么用本体：能快速用的不好复用，好复用的建不起

方案	周期短	投入少	可复用	可追溯	结论可信	
RAG 知识库	●●●	●●●	●●○	●○○	●○○	只管找到，不管对错与时效
提示词 / Dify / Skill	●●●	●●○	●○○	●○○	●●○	取决于写的人， 换个场景就失效
FIBO 类参考本体	●○○	●○○	●●●	●●●	●●○	定义最准，但测试场景不好用
场景轻量本体 （本方案）	●●○	●●○	●●○	●●●	●●●	带来源和版本，面向测试意图

●●● 好　●●○ 一般　●○○ 弱

本体的价值在最后一列：结论可信——带来源、可追溯、面向测试意图　但大而全的本体，建得起吗？

大而全的本体：建不起、等不起、调不动

行业里的大本体，都是这样建出来的

Cyc 1984 起
通用常识推理；150 万+概念、2500 万+公理

Gene Ontology 1998 起
基因功能注释；约 3.8 万术语（2026）

SNOMED CT 2002 发布
临床术语体系；36 万+概念，按月发布

FIBO 持续维护
金融业务语义；数百个本体文件，季度发布

共同点：长期建设、专家协作、持续维护

建设周期
以年计

需求周期
按周推进



行业联盟长期建设
多年投入、专家持续维护

测试现场的需求
项目按周排期，快速响应

测试项目的现实

等不起
项目按周排期，
需求下周就要测

调不动
跨团队协调专家，
难以随时到位

定不了
业务口径需要
协同确认

所以不做大而全：从一个测试场景切小，让本体跟着测试工作一点点长出来

切小怎么切：按知识的来源和节奏，分三层

一个测试场景里，其实混着三类知识——来源不同、变化节奏不同、维护的人也不同

业务知识



来自
业务需求
和规范

从哪来	需求、规范、接口
多久变	随需求版本变
谁维护	业务分析 + 测试
像什么	典型本体
例：超时专项	超时阈值 4 秒

测试方法



来自
专家经验

从哪来	专家经验
多久变	相对稳定
谁维护	测试专家
像什么	方法论
例：超时专项	边界值怎么取

执行知识



来自
APP、环境
和轨迹

从哪来	APP、环境、轨迹
多久变	随应用版本和环境变
谁维护	自动化与执行人员
像什么	知识图谱
例：超时专项	挡板延迟怎么配

硬塞进一个本体会怎样

- ✕ 需求一改，方法和执行一起受牵连
- ✕ 边界和维护责任说不清
- ✕ 只能一次建完，没法分段交付



贯穿测试流程：需求理解 → 测试分析 → 案例生成 → 案例执行

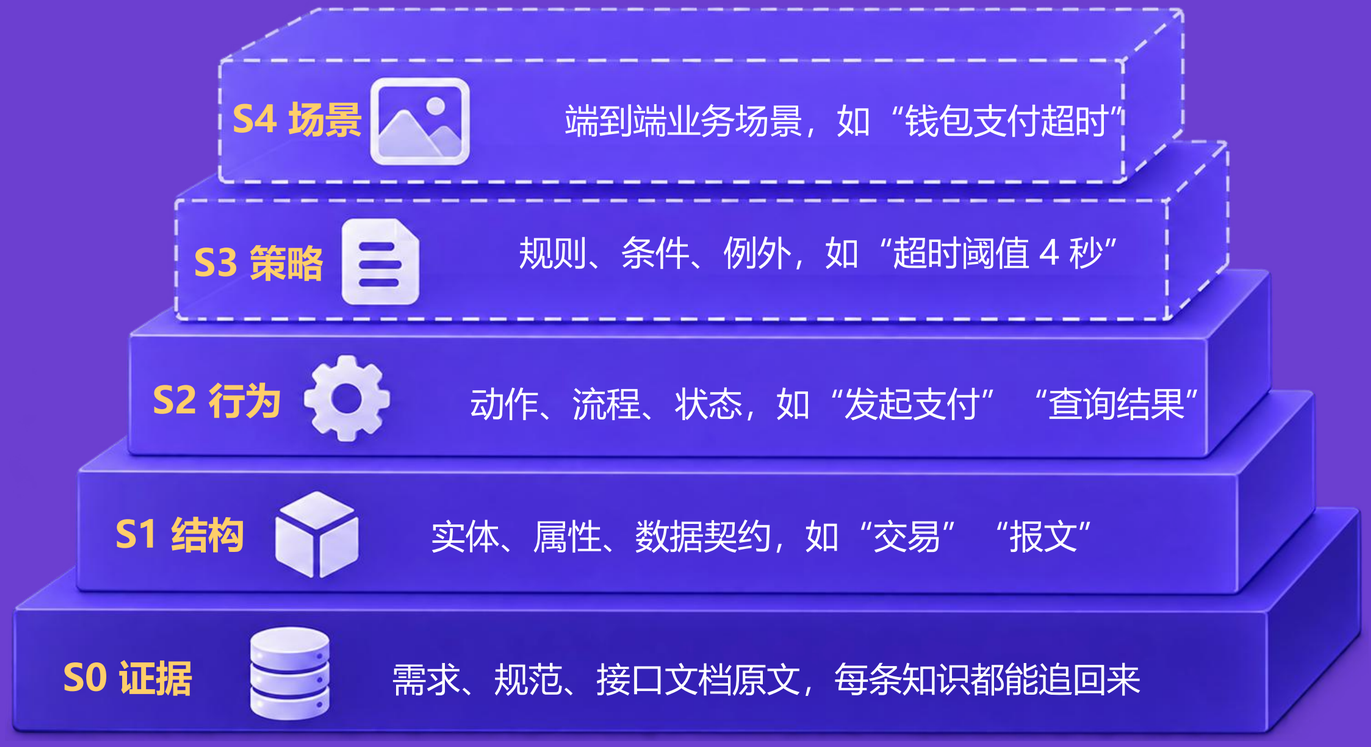
所以分三层：分层积累、各自维护，在测试流程中协同使用

接下来：怎么接、怎么建、怎么用

03 方法：测试工艺 × 分层本体



怎么建：业务本体按层长出来，AI 提候选、人来定



部分层或信息可以为空：不为填满层级而补造规则和场景

构建循环

AI 提取候选
从需求、规范、历史案例识别概念、关系、条件

工具校验
引用是否存在、类型是否兼容、有无冲突

人：校准与补建
确认、修正，也可手动新建实体、关系、属性

发布版本
带来源与版本号，可回溯

使用反馈 → 增量修订

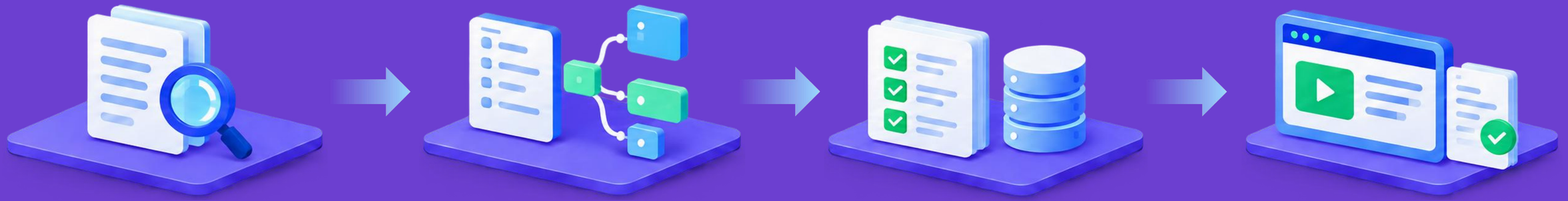
测试本体 经验和方法论为主
专家方法沉淀成工艺卡与知识包

业务本体 存量材料和事实为主
功能需求与设计、存量系统知识等

测试工艺：把测试拆成四道工序

需求理解 → 测试分析 → 案例生成 → 案例执行

业务本体 | 测试本体 | 应用本体



业务建模为主，持续演进扩充

澄清业务含义、明确被测要素、绑定测试数据

分析链路的起点，问题可追溯、可澄清

专家经验为主，稳定后不再演进

消费需求、业务本体，生成案例

设计有依据，案例可操作，问题可溯源

随应用演进的应用”地图”

Agent探索、Agent使用

弥补需求和开发落地的部分gap

每道工序写清：输入 • 方法 • 产出 • 检验 | 业务含义在生成案例前澄清，执行按案例与应用知识开展

完整语义与形式化需结合实际

第一版

完整语义归纳

参考 FIBO，十余个阶段逐层归纳
一个定义讨论半天、一个边界划分半天



第二版

规则全形式化

规则全部结构化，Test IR 也很重
维护成本上去了，模型反而没法用自然语言推理



- 维护成本上去了
- 没法用自然语言推理

现在

四句话

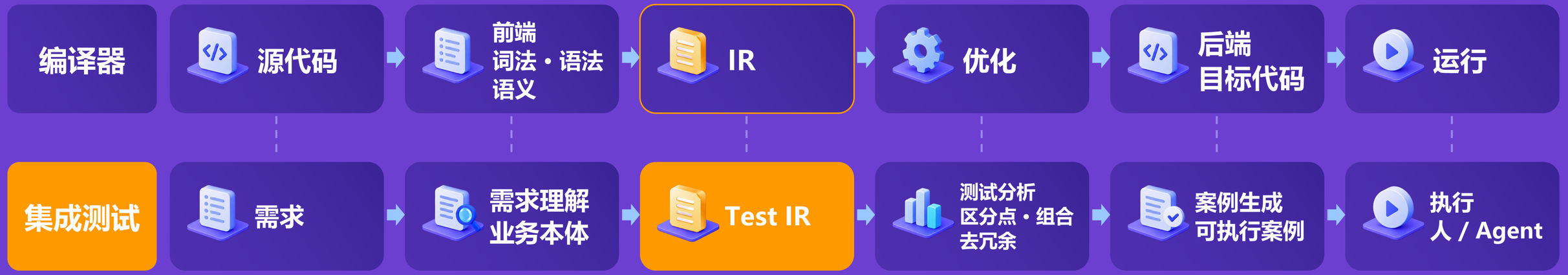


怎么判断要不要结构化

本机构处理超时阈值 4 秒	结构化（要确定性校验）
超时后最终结果以本机构处理为准	自然语言 + 来源 + 版本

本体刻画业务本质与规律；
面向任务按需提取，并随业务演进持续更新和维护。

Test IR：把需求“编译”成案例



传统上，这场“编译”全靠测试人员的脑子：需求读进去，案例写出来，再亲手执行

1 前后端解耦



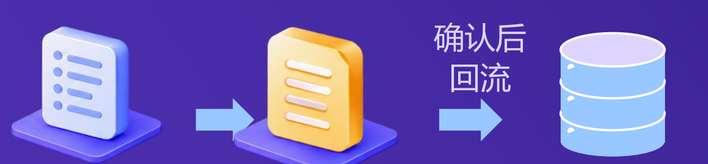
需求理解一次，分析、生成、执行都读同一份 IR；换执行方式不必重做分析

2 多个 Agent 的通用语言



事实、规则、条件、检查点、问题都在 IR 里，各阶段读写同一份记录，不传长文报告

3 隔离一次性信息



单次需求的事实先留在 IR，确认后才回流本体，不污染长期知识

怎么用：案例挂到本体上，变更沿关系传播

① 案例绑定本体

已有案例按测试意图挂到本体元素

② 解析变更需求

拆成事实与覆盖责任

③ 定位受影响元素

哪些对象、规则、行为变了

④ 沿关系找案例

精确引用 → 关系一跳 → 语义复核

⑤ 处置案例

修改 / 补充 / 原样 / 范围外

⑥ 变更回流本体

补充新规则，发布新版本



零命中 ≠ 无影响： 召回不到的也要标出来，交给人看

04 案例：从影响分析到案例生成



案例一：一份增量需求的影响分析

示意样例

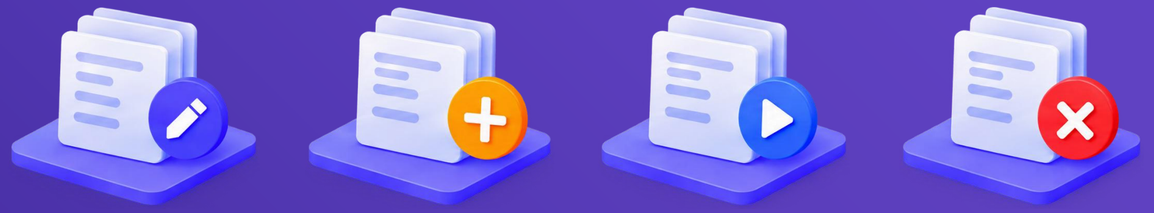
需求：支付类报文：超时阈值 4 秒 → 3 秒；超时提示改为“处理中”；超时后主动查询；迟到响应不重复记账；非支付类不变

从需求识别覆盖目标

已有变化	新增要求	保持不变
阈值变化、提示变化	主动查询、迟到响应不重复记账	非支付类规则

已有 10 条案例如何处置（示意）

3	2	4	1
修改后重跑	补充后重跑	原样重跑	范围外



仍需补齐的 2 项测试目标（示意）

#	测试目标	依据	现状	下一步
1	主动查询失效时如何处理	由主动查询要求推导的异常方向	预期行为待澄清	确认规则后补充检查点
2	迟到响应不重复记账	需求明确要求	现有案例缺少对应断言	补充不重复记账检查

案例处置按案例计数；覆盖缺口按测试目标计数，两者分别核对。

案例一：一份增量需求的影响分析

示意样例

流程示意



案例	变化	处置
超时专项-07	阈值 4→3 秒，提示改“处理中”	修改后重跑
超时专项-05	需补“30 秒内自动更新”断言	补充后重跑
查询-02	字面同为超时 4s，非支付类不变	原样重跑
设置-09	无结构影响路径	范围外

超时专项-07 的追踪链



案例二：让测试工艺贯穿需求到案例

理想状态 · 目标流程示意

业务本体：对象、关系、业务规则与来源

测试本体：设计方法、条件组合与覆盖关系

1. 需求进入



导入需求、
规范与业务背景

产物：可定位的原文

2. 识别问题与测试意图



发现歧义、缺失
和需验证的目标

产物：问题清单、测试意图

传递业务事实
与待澄清项

3. 分解规则与要素



保留条件、分支、
约束和依据

产物：规则、业务要素

4. 组合检查点



运用规则组合条件
补充独立检查

产物：覆盖目标、检查点

5. 自动生成案例



展开准备、操作
与逐步预期

产物：可理解、可操作的案例

影响追踪关系链（预期能力）

问题与产物保持关联

待澄清问题 → **业务规则** → **要素** → **检查点** → **案例**

需求澄清 → 定位关联内容、更新受影响模块

预期作用：需求未完备：问题可追溯，影响范围可识别 | 需求澄清后：快速定位相关影响，定向更新

案例二：让测试工艺贯穿需求到案例

需求与测试意图

规则原文

转账必须授权通过才能提交，否则交易报错

测试意图

验证授权对交易提交的控制

规则与要素

从规则中提取测试要素，形成组合依据

条件：通行方式 = 转账

要素：授权结果：通过 / 未通过

单笔金额：超限 / 不超限

测试本体指导要素分解与检查点设计

组合检查点

转账 × 授权通过：满足授权条件

转账 × 授权未通过：交易报错

测试案例组合

其他提交条件另行核对

案例节选：转账授权未通过拒绝

准备　人民币 5000 元转账，付款账户正常、余额充足，支付条件可验证通过

关键步骤

1. 进入跨行存款并选择转账
2. 刷卡或存折读取付款账号
3. 录入收款账号、人民币及 5000 元
4. 提交交易
5. 授权环节选择不通过

预期：交易报错，回显授权未通过的失败原因

未完备需求的影响追踪（目标）

问题　日累计按客户还是按账户计算?

累计金额要素

限额检查点

关联案例的数据准备

澄清后，沿关联定位并更新

明确的规则形成设计；未明确的问题保留影响关系。

05 总结：本体是手段不是目的



本体是手段，不是目的

先有场景，再建模

围绕任务提取对象、关系与业务规律



业务场景　　本体建模　　解决问题

AI 提候选，专家校准

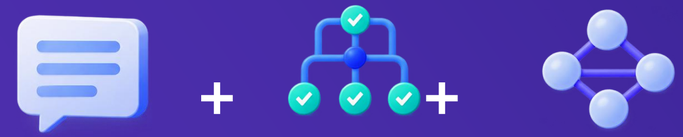
阈值、口径、适用范围与例外，需要业务依据



语义提取　　**语义校准**　　知识使用

不能全靠人建，也不能完全指望AI自动化

表达方式由任务决定



自然语言　　结构化规则　　关系



需要确定性结论时，
使用规则校验或专用推理

本体是当前弥补 LLM 领域业务知识不足的一种解法，未必是最优解。

本体知识 ? 领域模型 ? 领域超级智能体

能否逐步融合，仍待探索

要不要上本体?



Q1

同一个问题，每来一个新人
都要问一遍?

知识会被反复用 → 值得沉淀



Q2

需求一改，没人说得清该重测哪些?

有变更分析的刚需 → 值得建关系



Q3

老师傅一休假，就没人会
造这个异常?

经验只在人脑里 → 值得写成工艺

落地需要哪些基础

需要什么	解决什么
可定位的业务材料	找得到判断依据
可遵循的工艺规则	方法能够稳定复用
可确认口径的 业务与测试人员	处理歧义和 知识修订
用于实测的代表样例	判断结果是否正确

知识反复使用、关系复杂、经验依赖明显时，可从一个高价值场景试起
一次性、规则简单、复用价值低时，先完善文档与流程

领域知识工程的快速实践地图

基于评估与反馈，持续迭代



1

选定目标与边界

明确要支持的理解、判断或行动
选一个有代表性的真实场景

2

工艺规则化并实测

把专家方法拆成可遵循的规则
用真实样例检验步骤与结果

3

按需沉淀领域知识

记录对象、关系、规则及来源
AI 提出候选，专家校准

4

持续评估与迭代

用新场景和变化检验适用性
根据错误与反馈修订工艺和知识

判断有没有用： 结果正确性 · 覆盖完整性 · 依据可追溯性 · 人工修订成本

能运行、结果正确、能够复用，分别评估

回到那条案例

交易一致性 | 超时专项-07

① 前置：处理时间“较长”

② APP 发起，本机构发报文

③ 本机构 4 秒内未返回

④ APP 显示交易超时

⑤ 查看最终结果



用户发起交易 | 本机构处理并发报文 | 等待本机构结果... | 提示交易超时 | 查询交易记录

“较长”多长？从哪一刻起算？ → **业务本体**：澄清计时边界与阈值，确认后写成规则

这个异常怎么造？ → **应用本体**：按确认的链路构造异常，写进操作地图

阈值改成 3 秒呢？ → **变更分析**：沿本体关系，定位受影响的案例

持续追求的，是更准确的业务理解、更可靠的判断和行动。

无论未来采用何种技术，每一次测试，都可以为下一次留下依据。

THANKS



欢迎提问交流